

News Headline Classifier

Team Members: Yingyu Ma, Jiheng Hou, Mingang Guo

Project: News Source

1 Introduction

We address Project B, which asks us to classify whether a news headline comes from Fox News or NBC News. The official metric is accuracy, and the backend submission contract requires `preprocess.py`, `model.py`, and optionally `model.pt`. Our final code submission follows that contract and reached an observed Hugging Face leaderboard accuracy of **0.8742**, compared with the course baseline accuracy of 0.6649.

The main challenge is that the starter data are URLs, while the meaningful input signal should be the article headline rather than source-leaking URL tokens. We therefore built a scraping and cleaning pipeline, trained headline-only classifiers, and used exploratory experiments to show that URL text can create misleading validation scores. The final repository is available on [GitHub](#), and the collected dataset is published on [Hugging Face](#) for reproducibility, including the scraping pipeline, cleaned headline data, model files, and report artifacts.

2 Core Components

Data collection and curation.

Base scrape. The project description lists 3,815 starter URLs, but the downloaded starter file in our repository contains 3,805 rows: 2,000 Fox News URLs and 1,805 NBC URLs. We wrote `Newsheadlines/scrape_headlines.py` to recover article headlines and labels. Labels are inferred from URL hostnames, with Fox News mapped to 0 and NBC mapped to 1. The scraper uses browser-like request headers, retries, request delays, canonical URL variants, metadata extraction from Open Graph/Twitter title fields, site-specific and generic `h1` selectors, and Wayback/Jina fallbacks. It saved 3,805 raw rows with no failed rows: 3,801 direct HTML headlines, 1 direct-variant headline, 2 Wayback headlines, and 1 URL-slug fallback.

Cleaning. We normalized HTML entities, quotes, whitespace, and newlines, removed duplicate URL and headline records, dropped invalid or very short headlines, and checked label/source consistency. Two duplicate headline-label rows were removed, leaving 3,803 cleaned base examples. For headline-only validation experiments, we also removed the single URL-slug fallback row, leaving 3,802 examples. This prevents URL-derived pseudo-text from being treated as a real headline.

External augmentation. To improve hidden-test generalization, we also collected current public Fox/NBC headlines from RSS feeds, sitemaps, and section pages. The compliant external pool contains 2,453 non-duplicate, cleaned headlines. Since that pool is NBC-heavy, the final training artifact uses a balanced augmented training set with 4,446 rows: 2,218 Fox examples and 2,228 NBC examples. URL strings are retained only for provenance and, in the backend setting, for fetching the article headline; URL domains, paths, ids, and source names are not used as classifier features.

Dataset stage	Rows	Fox	NBC	Role
Starter URLs	3,805	2,000	1,805	Input to scraper
Cleaned base scrape	3,803	2,000	1,803	Main collected dataset
Headline-only base	3,802	2,000	1,802	Internal validation and report analysis
Compliant external pool	2,453	324	2,129	Additional public headlines
Final augmented train set	4,446	2,218	2,228	Final packed model artifact

Table 1: Dataset construction summary.

Model design and iterations. We treated model development as a sequence of task-aligned leaderboard

experiments rather than a single local validation sweep. The main constraint throughout the iteration was compliance with the Project B setting: the model should classify from real headline text, not from URL domains, URL paths, article ids, or other source-leaking strings. Therefore, URLs were used only as an entry point for scraping the article headline and, when necessary, for constructing evaluator labels; URL text itself was not returned as an input feature.

Our first working submission used a lightweight sparse-text model and achieved 0.8175 accuracy on the Hugging Face `url_val` leaderboard. After fixing the preprocessing path so that URL-only rows were converted into scraped headline text instead of leaving raw URL-like tokens in the input, we built a stronger headline-only sparse ensemble. This version combined word/character TF-IDF Logistic Regression with a low-rank DLR/SVD/RBF-style branch, giving a stable 0.8425 leaderboard accuracy. We then made several small but important refinements: stricter source-token masking, better handling of failed URL fetches, and small changes to the character and DLR branch weights. These changes produced incremental gains to 0.8442 and then 0.8458.

A larger improvement came from threshold calibration. The default 0.5 probability threshold was not optimal for the hidden validation distribution, especially because NBC and Fox errors were not symmetric. We therefore performed a narrow threshold sweep around the best sparse ensemble. Raising the positive-class threshold to approximately 0.5250 improved leaderboard accuracy from the mid-0.84 range to 0.8642. Additional weight, gamma, and threshold micro-grids around this configuration mostly plateaued: several variants matched 0.8642, but none reliably exceeded it. This suggested that the LR/DLR soft-vote family had reached a local ceiling.

To break this plateau, we added a more heterogeneous branch rather than continuing to tune only the same LR/DLR model family. We tested Complement Naive Bayes and calibrated SGD classifiers as headline-only auxiliary models. Individually, these branches were weaker than the current best ensemble: ComplementNB alone reached about 0.8608, the calibrated SGD branch was much lower, and the heterogeneous branch alone was also below the best model. However, their error patterns were different enough to help when blended with the existing LR/DLR ensemble. The final submitted model, `cur_nb_sgdcals5525`, combines the current LR/DLR ensemble with ComplementNB and calibrated SGD probability estimates, then applies a tuned threshold of 0.5525. This produced our best observed leaderboard accuracy, 0.8742.

Iteration	HF Accuracy	Main change
Initial KK model	0.8175	First working headline classifier submission
URL-fetch sparse ensemble	0.8425	Real headline extraction plus word/char LR and DLR-style branch
DLR-enhanced ensemble	0.8442	Adjusted DLR/character branch contribution
Preprocessing and calibration refinement	0.8458	Better source masking and threshold-ready probability behavior
High-threshold LR/DLR ensemble	0.8642	Raised positive-class threshold to around 0.5250
Current + NB + calibrated SGD	0.8733	Added heterogeneous headline-only auxiliary branches
Final submitted model	0.8742	Same heterogeneous blend with threshold 0.5525

Table 2: Major model iterations on the Hugging Face `url_val` leaderboard. Many smaller weight, gamma, and threshold variants were tested but omitted here because they either matched or underperformed these milestones.

Several attempted directions were rejected. SVM-style stacking, a small transformer experiment, and pure SGD-style classifiers did not improve the leaderboard score enough relative to their added complexity or runtime. The final design therefore remains deliberately lightweight: it uses only headline text, runs quickly in the course backend, and benefits from complementary sparse classifiers rather than relying on URL shortcuts.

Backend interface. The wrapper exposes `Model` and `get_model()` through `model.py`. The preprocessing step accepts headline columns when present. In the URL-only backend setting, `prepare_data` uses URLs only to fetch article titles or headlines before prediction. For local evaluation scripts, labels are kept only for scoring when available; they are not returned as model input and are not used by the classifier. Raw URL text, domains, paths, and article ids are never returned as classifier features. Explicit outlet names such as “Fox News”, “NBC News”, “MSNBC”, and “TODAY” are masked during normalization to reduce label leakage.

Evaluation protocols and results. Our primary metric is accuracy, matching the official leaderboard. We also used macro F1 on internal splits to monitor class balance. All internal validation results below use a stratified 80/20 split with random seed 42 unless otherwise noted. The final HF result is the observed hidden leaderboard accuracy.

Model or setting	Accuracy	Macro F1	Notes
Course baseline	0.6649	0.66	TF-IDF 100 + Logistic Regression, from project document
Word TF-IDF LR	0.8003	0.7991	3,803-row cleaned base split; <code>train_news_b_v1.py</code>
Char 3–5g TF-IDF LR	0.8305	0.8292	3,802-row headline-only split; strongest single model
Final ensemble, local clean sanity check	0.9437	–	Full cleaned base corpus; not a held-out score
Final ensemble, Hugging Face leaderboard	0.8742	–	Official hidden accuracy signal

Table 3: Main model results. The local full-corpus value is a compatibility sanity check, not a model-selection metric.

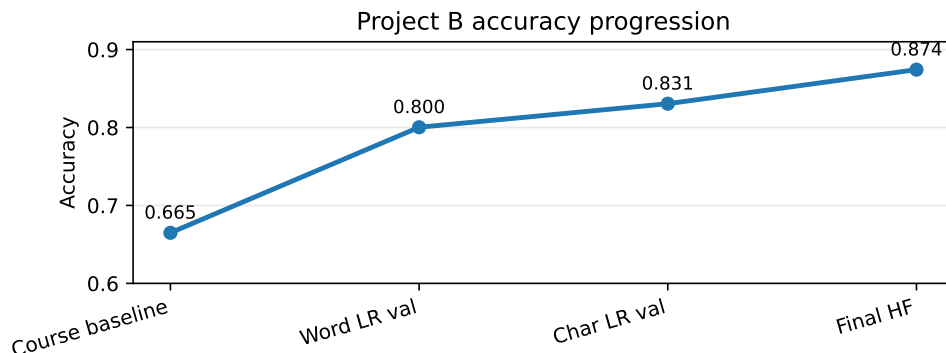


Figure 1: Accuracy progression from the course baseline to the final Hugging Face submission.

The leaderboard result shows a substantial improvement over the course baseline, with an absolute gain of about 20.9 percentage points. We did not treat the local full-corpus accuracy as a hidden-test proxy because the final artifact is trained on the cleaned base data and selected external headlines; it is useful only for checking that `preprocess.py`, `model.py`, and `model.pt` work end to end.

The final leaderboard number is the observed Hugging Face result recorded in the repository README. The course baseline comes from the project handout, and the Word LR validation row comes from the deliverables metric table generated by `train_news_b_v1.py`.

3 Exploratory Components

Analysis: shortcut risk. Our exploratory question was: *Do URL-derived representations create shortcut behavior that does not transfer to headline-only inference?* This matters because the starter data are URLs, and URLs contain outlet-specific domains, section names, path structures, article ids, and slug words. A model can appear strong if both training and validation expose these URL patterns, but that does not mean it has learned source style from headlines.

We compared five train/test interfaces using the same split and a consistent word-feature pipeline. Headline-to-headline training achieved 0.8003 accuracy. URL-to-URL evaluation reached 0.9671, but this is a shortcut-prone proxy task. When the URL-trained model was evaluated on real headlines, accuracy dropped to 0.5440. Similarly, a headline+URL model achieved 0.9566 when both fields were available, but fell to 0.6531 when only headlines were provided at inference.

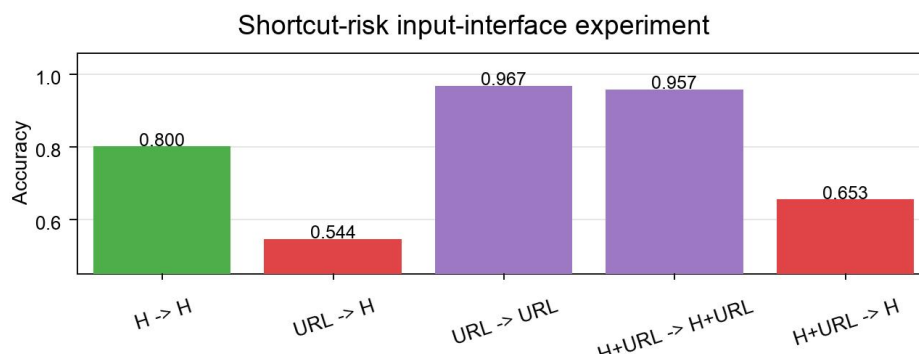


Figure 2: Shortcut-risk input-interface experiment. H denotes real headline text. URL-based representations achieve high accuracy when URL-derived patterns are available during both training and testing, but their performance drops substantially when evaluated on headline-only inputs. This indicates that URL features can create shortcut behavior rather than robust headline-style classification, and supports the final decision to use URLs only as retrieval handles for article headlines.

This analysis directly changed our final design. We rejected raw URL-string models even when their validation numbers looked high. The final submission instead uses URLs only as retrieval handles for real headline text, then masks obvious outlet names and removes URL-like tokens before classification. A second cleaning stress test also supported strict cleaning: after injecting conflicting duplicate rows, loose cleaning dropped validation accuracy to 0.7595, while strict deduplication and invalid-headline filtering recovered 0.8003.

Limitations and error analysis. The main limitation is that source classification from headlines is partly a style task and partly a topic-distribution task. Some Fox and NBC headlines cover the same national, political, crime, health, and world-news events, so headline text alone can be genuinely ambiguous. We therefore avoided claiming that the model learns only writing style; it likely learns a mixture of topic, named entities, phrasing, and short-form editorial conventions. Internal class-balanced metrics helped monitor this, but only the hidden leaderboard directly measures performance on the course distribution.

The second limitation is temporal drift. The external augmentation was collected after the starter scrape from public feeds and section pages. This helped the hidden score when selected carefully, but it also made validation noisier because current-news topics can be overrepresented. To reduce this risk, we used a balanced final training subset rather than the entire NBC-heavy external pool, masked explicit outlet names, and avoided hard-coding topics or source-specific URL patterns.

Finally, the backend URL-only setting introduces an operational risk: some article pages may block requests, change markup, or time out. Our preprocessing code mitigates this with browser-like headers, parallel but bounded fetching, metadata-first headline extraction, and a neutral missing-headline placeholder. If a headline cannot be fetched, the model receives no URL slug, domain, or article id signal. This preserves compliance with our exploratory finding, even though it may sacrifice some examples when pages are unavailable.

Reproducibility. The final upload files are `model.py`, `preprocess.py`, and `model.pt` at the repository root. Backup copies and experiment artifacts are stored under `submission_hf_urlfetch/`, `experiments/`, and `deliverables/`. We checked the local backend-style interface with:

```
python Newsheadlines/eval_project_b.py \
  --model model.py \
  --preprocess preprocess.py \
  --csv <csv>
```

4 Team Contributions

Yingyu Ma led report writing and organization, including synthesizing dataset construction, model iterations, exploratory findings, and final LaTeX presentation.

Jiheng Hou led the code implementation, including scraping, preprocessing, model training, leaderboard-oriented model iteration, evaluation, and backend compatibility.

Mingang Guo contributed experiment discussion, error-analysis review, external-data validation, and figure/table/result checking for the final report.

5 Extra Credit

We submitted a short extra-credit project video with the final materials. Video link: <https://drive.google.com/file/d/12gdfQJgobv5Z01hMkZPMEeCEEQmRqwzL/view?usp=sharing>.

6 Acknowledgments

We did not receive substantial project-specific help from outside the project team beyond the CIS 4190/5190 project handout, starter materials, baseline model, general course guidance, and publicly available documentation for standard Python machine learning and web-scraping libraries. These resources helped us understand the required submission format, evaluation protocol, and implementation details for data collection, preprocessing, model training, and backend compatibility.

We used AI-assisted tools in a limited way for debugging support, wording suggestions, and report editing. These tools were used to clarify implementation details and improve presentation, but not to replace project design, data collection, model selection, experimental analysis, or final verification. All reported results and conclusions were reviewed and validated by the team.

7 Suggestions for future iterations of this project

Future work would focus on building a more systematic and reliable external-data selection procedure. In our current pipeline, balanced external augmentation improved the hidden leaderboard score, but local validation was sometimes noisy relative to Hugging Face results. A stronger next step would be to create a frozen, time-split validation set from later Fox and NBC headlines. This would make it easier to test whether new external data actually improves generalization, rather than only matching the current leaderboard distribution.

Another useful improvement would be a more careful study of which external examples should be included. Instead of adding all collected headlines or manually choosing a balanced subset, future versions could filter examples based on headline quality, publication date, section, length, and similarity to the starter data. This may reduce topic drift, especially when current news contains many repeated stories from one outlet. We could also compare different augmentation ratios to see whether the model benefits more from balance, recency, or diversity.

A second direction would be improving model calibration and threshold selection. Our final result benefited from threshold tuning, which suggests that the default probability cutoff was not ideal for the hidden data distribution. In a future version, we would tune thresholds using a cleaner validation protocol and examine class-specific errors more carefully. For example, we could separately analyze Fox-to-NBC and NBC-to-Fox mistakes to understand whether errors come from ambiguous headlines, topic overlap, or differences in headline style.

Finally, future iterations could make the backend preprocessing more robust. Since the official input may contain URLs, the model depends on successfully fetching or extracting real article headlines. Some pages may block requests, change their HTML format, or become unavailable over time. A stronger version of the

pipeline could cache extracted headlines, add more reliable metadata parsing, and record failure cases more explicitly. This would make the submission easier to reproduce and reduce the risk that backend performance changes because of page-fetching issues rather than model quality.

8 Plots and appendix

These supplemental plots provide additional visual checks for the cleaning, shortcut-risk, and base scraping results discussed in the main report.

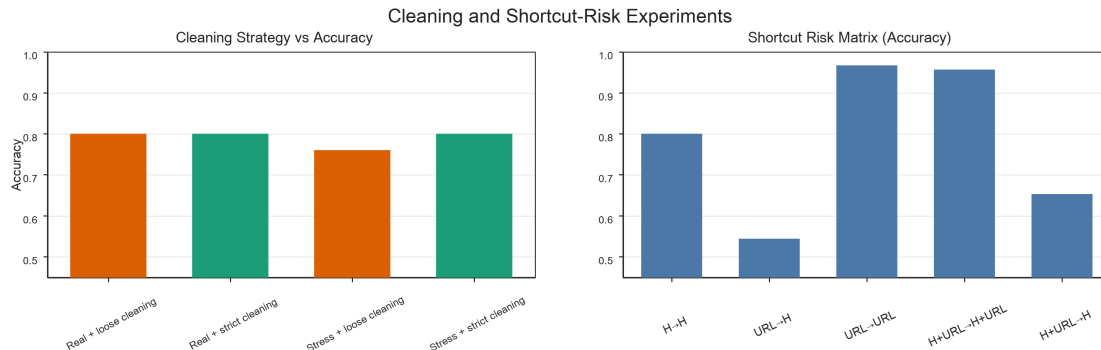


Figure 3: Additional checks for cleaning and shortcut risk. Strict cleaning gave similar accuracy on the normal cleaned data, but it performed better in the stress-test setting where noisy or conflicting examples were added. The shortcut-risk results show that models using URL text can look very strong when URL patterns are available during both training and testing, but their performance drops when they are tested on real headlines only. This supports our decision to use URLs only for headline retrieval, not as classifier features.

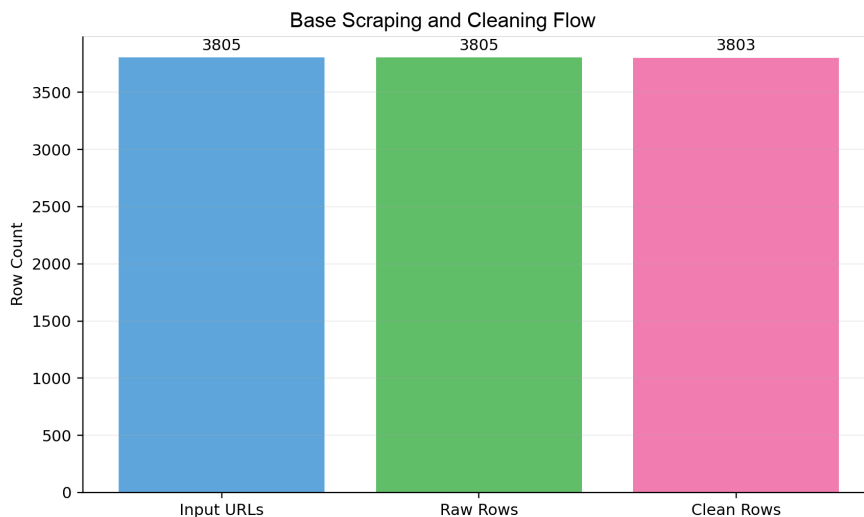


Figure 4: Base scraping and cleaning flow for Project B. The input URLs were scraped into raw headline rows and then cleaned to remove duplicate or invalid examples. The small decrease from raw rows to clean rows shows that most collected examples were usable, while the cleaning step mainly removed problematic records before model training and validation.